관 인 생 략

출 원 번 호 통 지 서

출 원 일 자	2024.05.02
특 기 사 항	심사청구(유) 공개신청(무)
출 원 번 호	10-2024-0058692 (접수번호 1-1-2024-0482533-16) (DAS접근코드15E6)
출원인 명칭	충북대학교 산학협력단(2-2004-016822-6)
대리인 성명	특허법인무한(9-2007-100061-4)
발명자 성명	팜칸콴 김태홍
발명의 명칭	수직적 포드 오토 스케일러를 사용한 탄력적 연합 학습 방법 및 시스템

특 허 청 장

<< 안내 >>

1. 귀하의 출원은 위와 같이 정상적으로 접수되었으며, 이후의 심사 진행상황은 출원번호를 이용하여 특허로 홈페이지(www.patent.go.kr)에서 확인하실 수 있습니다.
2. 출원에 따른 수수료는 접수일로부터 다음날까지 동봉된 납입영수증에 성명, 납부자번호 등을 기재하여 가까운 은행 또는 우체국에 납부하여야 합니다.
※ 납부자번호 : 0131(기관코드) + 접수번호
3. 귀하의 주소, 연락처 등의 변경사항이 있을 경우, 즉시 [특허고객번호 정보변경(경정), 정정신고서]를 제출하여야 출원 이후의 각종 통지서를 정상적으로 받을 수 있습니다.
4. 기타 심사 절차(제도)에 관한 사항은 특허청 홈페이지를 참고하시거나 특허고객상담센터(☎ 1544-8080)에 문의하여 주시기 바랍니다.
※ 심사제도 안내 : https://www.kipo.go.kr-지식재산제도

9200710006141011101000033550000000

특허출원서

【출원구분】 특허출원

【출원인】

【명칭】 충북대학교 산학협력단

【특허고객번호】 2-2004-016822-6

【대리인】

【명칭】 특허법인무한

【대리인번호】 9-2007-100061-4

【지정된 변리사】 구기완

【포괄위임등록번호】 2017-033491-9

【발명의 국문명칭】 수직적 포드 오토 스케일러를 사용한 탄력적 연합 학습 방법 및 시스템

【발명의 영문명칭】 ELASTIC FEDERATED LEARNING METHOD AND SYSTEM WITH VERTICAL POD AUTOSCALER

【발명자】

【성명의 국문표기】 팜칸콴

【성명의 영문표기】 Pham Khanh Quan

【주민등록번호】 970901-5000000

【우편번호】 28644

【주소】 충청북도 청주시 서원구 충대로 1, 제5공학관 307호 (개신동, 충북대학교)

【발명자】

【성명의 국문표기】 김태홍

【성명의 영문표기】 KIM Tae Hong

【주민등록번호】 820122-1000000

【우편번호】 28644

【주소】 충청북도 청주시 서원구 충대로 1, 제5공학관 403호 (개신동, 충북대학교)

【출원언어】 국어

【심사청구】 청구

【공지예외적용대상 증명서류의 내용】

【공개형태】 논문 발표

【공개일자】 2024.05.01

【이 발명을 지원한 국가연구개발사업】

【과제고유번호】 1345363699

【과제번호】 2022R1I1A3072355

【부처명】 교육부

【과제관리(전문)기관명】 한국연구재단

【연구사업명】 이공학학술연구기반구축

【연구과제명】 엣지 컴퓨팅 환경에서의 연합학습 기술 개발 및 검증

【과제수행기관명】 충북대학교

【연구기간】 2023.03.01 ~ 2024.02.29

【이 발명을 지원한 국가연구개발사업】

【과제고유번호】 1711193777

【과제번호】 2020-0-01462-004

【부처명】 과학기술정보통신부

【과제관리(전문)기관명】 정보통신기획평가원

【연구사업명】 정보통신방송혁신인재양성

【연구과제명】 Grand ICT 연구센터(충북대)

【과제수행기관명】 충북대학교산학협력단

【연구기간】 2023.01.01 ~ 2023.12.31

위와 같이 특허청장에게 제출합니다.

대리인 특허법인무한 (서명 또는 인)

【수수료】

【기본출원료】	0	면	46,000	원
【가산출원료】	34	면	0	원
【우선권주장료】	0	건	0	원
【심사청구료】	9	항	625,000	원
【합계】		671,000원		
【감면사유】	전담조직(50%감면)[1]			
【감면후 수수료】			335,500	원

【첨부서류】 1.공지예외적용대상(신규성상실의예외, 출원시의특례)규정을 적용받기 위한 증명서류_1통

【발명의 설명】

【발명의 명칭】

수직적 포드 오토 스케일러를 사용한 탄력적 연합 학습 방법 및 시스템{ELASTIC FEDERATED LEARNING METHOD AND SYSTEM WITH VERTICAL POD AUTOSCALER}

【기술분야】

<0001> 본 발명은 연합 학습 방법 및 장치에 관한 것으로, 보다 구체적으로는 수직적 포드 오토 스케일러를 사용하여 리소스를 탄력적으로 할당하는 연합 학습 방법 및 장치에 관한 것이다.

【발명의 배경이 되는 기술】

<0002> 연합 학습(FL)은 기존의 중앙 집중식 모델 훈련 방법에 내재되어 있는 개인 정보 노출 및 보안 위험을 해결하기 위해 제안된 기술로, 모델 학습을 위해 데이터를 중앙 집중화해야 하는 기존 접근 방식과 달리 분산된 엣지 디바이스 전반에서 협업 학습을 가능하게 한다.

<0003> 즉, 연합 학습은 민감한 데이터를 개별 디바이스에 보관함으로써 중앙 집중식 데이터 집계와 관련된 개인 정보 보호 문제를 최소화하여 데이터 프라이버시 및 보안을 강화할 수 있다.

<0004> 그러나, 동적이고 분산된 엣지 환경 내에서 리소스 활용을 최적화하기 위해서는 시스템 이질성 및 데이터 이질성을 고려한 설계가 필요하다.

<0005> 시스템 이질성(System heterogeneity)은 엣지 노드의 컴퓨팅 리소스 (처리

용량, 메모리, 스토리지 등)의 다양성을 의미한다. 엣지 노드는 센서, 스마트폰, IoT(사물 인터넷) 디바이스 또는 엣지 서버가 될 수 있으며, 각각 별도의 컴퓨팅 리소스와 통신 기능을 갖추고 있다. 예를 들어 리소스가 낮은 노드는 리소스가 높은 노드에 비해 트레이닝 작업을 완료하는 데 시간이 더 오래 걸릴 수 있으므로, 연합 학습 시스템 전체에서 트레이닝 시간 차이가 발생할 수 있다.

<0006> 데이터 이질성(Data heterogeneity)은 네트워크의 여러 노드에서 사용 가능한 데이터가 독립적, 균등하지 않게 배포된 (Non-independent Identically Distributed) 환경을 의미한다. 즉, 클라이언트 간의 학습 샘플 수가 크게 다를 경우, 모델의 학습 성능에 잠재적으로 영향을 줄 수 있다. 예를 들어, 데이터가 많은 노드는 더 많은 컴퓨팅 리소스를 필요로 하므로 특히 리소스가 제한된 환경에서는 연합학습의 전반적인 성능 저하에 영향을 미칠 수 있다. 또한, 노드 리소스의 가변성으로 인해 일부 노드는 활용도가 낮은 반면 다른 노드는 과도하게 활용되는 리소스 사용률의 불균형이 발생할 수 있다.

<0007> 따라서, 엣지 컴퓨팅 환경에서의 시스템 이질성 및 데이터 이질성 문제를 해결할 수 있는 연합 학습 방법이 요청되고 있다.

【발명의 내용】

【해결하고자 하는 과제】

<0008> 본 발명은 시계열 과거 데이터(과거 리소스 사용률 정보)를 분석하고, 최적화 기법을 활용하여 실제 애플리케이션 수요에 따라 컨테이너 기반 애플리케이션 서비스에 할당된 리소스를 동적으로 조정함으로써, 연합 학습의 처리 속도를 개선

하는 방법 및 장치를 제공한다.

【과제의 해결 수단】

<0009> 본 발명의 일실시예에 따른 연합 학습 방법은 FL(Federated Learning) 포드의 과거 리소스 사용률 정보에 기초하여 상기 FL 포드의 권장 리소스 요청을 결정하는 단계; 상기 FL 포드의 현재 리소스 요청과 상기 권장 리소스 요청 간의 차이를 식별하는 단계; 및 상기 현재 리소스 요청과 상기 권장 리소스 요청 간의 차이가 임계값을 초과하는 경우, 상기 FL 포드의 리소스 요청 및 리소스 제한을 상기 권장 리소스 요청에 따라 업데이트하는 단계를 포함할 수 있다.

<0010> 본 발명의 일실시예에 따른 연합 학습 방법의 상기 권장 리소스 요청을 결정하는 단계는, 리소스 정보를 정렬하는 단계; 정렬된 리소스 정보를 기초로 목표 추정치를 결정하는 단계; 상기 과거 리소스 사용률 정보에 기초하여 최소 리소스를 결정하는 단계; 상기 목표 추정치가 최소 리소스를 초과하는 경우, 상기 목표 추정치를 상기 권장 리소스 요청의 권장 설정으로 결정하는 단계; 및 상기 목표 추정치가 최소 리소스 이하인 경우, 상기 최소 리소스를 상기 권장 리소스 요청의 권장 설정으로 결정하는 단계를 포함할 수 있다.

<0011> 본 발명의 일실시예에 따른 연합 학습 방법의 상기 목표 추정치를 결정하는 단계는, 기 지정된 목표 백분위수에서 기록 리소스값을 선택하는 단계; 및 선택한 기록 리소스값과 안전 마진을 연산하여 상기 목표 추정치를 결정하는 단계를 포함할 수 있다.

<0012> 본 발명의 일실시예에 따른 연합 학습 방법의 상기 최소 리소스를 결정하는

단계는, 상기 과거 리소스 사용률 정보에서 가장 낮은 CPU값 및 RAM값을 검색하는 단계; 및 검색한 CPU값 및 RAM값을 최소 리소스로 결정하는 단계를 포함할 수 있다.

<0013> 본 발명의 일실시예에 따른 연합 학습 시스템은 FL 애플리케이션의 실행을 관리하는 FL 포드가 포함된 FL 엣지 노드; 및 상기 FL 포드의 리소스를 관리하는 FL 클라우드 노드를 포함하고, 상기 FL 클라우드 노드는, 상기 FL 포드의 과거 리소스 사용률 정보에 기초하여 상기 FL 포드의 권장 리소스 요청을 결정하고, 상기 FL 포드의 현재 리소스 요청과 상기 권장 리소스 요청 간의 차이를 식별하며, 상기 현재 리소스 요청과 상기 권장 리소스 요청 간의 차이가 임계값을 초과하는 경우, 상기 FL 포드의 리소스 요청 및 리소스 제한을 상기 권장 리소스 요청에 따라 업데이트할 수 있다.

<0014> 본 발명의 일실시예에 따른 연합 학습 시스템의 상기 FL 엣지 노드는, 상기 FL 애플리케이션의 실행을 관리하고, 리소스 할당을 처리하는 FL 포드 및 상기 FL 포드에서 VPA(Vertical Pod Autoscaler) 동작을 구성하고, 상기 FL 포드를 모니터링하여 상기 FL 포드의 상태 정보를 관리하는 VPA API 오브젝트를 포함할 수 있다.

<0015> 본 발명의 일실시예에 따른 연합 학습 시스템의 상기 FL 클라우드 노드는, 상기 FL 포드에서 상기 FL 포드의 리소스 정보를 수집하여 히스토리 스토리지에 저장하는 매트릭 서버; 저장된 리소스 정보들을 과거 리소스 사용률 정보로 나타내는 히스토리 스토리지; VPA API 객체를 기반으로 신규 FL 포드 생성 요청에 상기 권

장 리소스 요청을 적용하거나, 기존 FL 포드의 리소스 요청 및 리소스 제한을 상기 권장 리소스 요청에 따라 업데이트하는 VPA 어드미션 플러그인; 및 상기 FL 포드의 권장 리소스 요청을 결정하는 VPA 컨트롤러를 포함할 수 있다.

<0016> 본 발명의 일실시예에 따른 연합 학습 시스템의 상기 VPA 컨트롤러는, 상기 FL 엣지 노드의 VPA API 오브젝트의 리소스 상태를 모니터링하고, 상기 VPA API 오브젝트의 리소스 상태를 업데이트 설정(Set recommendation)할 수 있다.

<0017> 본 발명의 일실시예에 따른 연합 학습 시스템의 상기 VPA 어드미션 플러그인은, 상기 FL 엣지 노드의 VPA API 오브젝트의 리소스 상태를 모니터링할 수 있다.

【발명의 효과】

<0018> 본 발명의 일실시예에 의하면, 시계열 과거 데이터(과거 리소스 사용률 정보)를 분석하고, 최적화 기법을 활용하여 실제 애플리케이션 수요에 따라 컨테이너 기반 애플리케이션 서비스에 할당된 리소스를 동적으로 조정함으로써, 연합 학습의 정확도를 높이고 처리 속도를 개선할 수 있다.

【도면의 간단한 설명】

<0019> 도 1은 본 발명의 일실시예에 따른 연합 학습 시스템을 도시한 도면이다.

도 2는 본 발명의 일실시예에 따른 연합 학습 시스템의 클라우드 노드와 엣지 노드의 상세 구성 및 동작을 나타내는 도면이다.

도 3은 본 발명의 일실시예에 따른 과거 리소스 사용률 정보 검색 과정의 조일례이다.

도 4는 본 발명의 일실시예에 따른 리커맨더 함수, 및 업데이터 함수의 일례

이다.

도 5는 본 발명의 일실시예에 따른 연합 학습 방법을 도시한 플로우차트이다.

도 6은 본 발명의 일실시예에 따른 연합 학습 방법의 권장 리소스 요청 결정 과정을 도시한 플로우차트이다.

【발명을 실시하기 위한 구체적인 내용】

<0020> 이하에서, 첨부된 도면을 참조하여 실시예들을 상세하게 설명한다. 그러나, 실시예들에는 다양한 변경이 가해질 수 있어서 특허출원의 권리 범위가 이러한 실시예들에 의해 제한되거나 한정되는 것은 아니다. 실시예들에 대한 모든 변경, 균등물 내지 대체물이 권리 범위에 포함되는 것으로 이해되어야 한다.

<0021> 실시예에서 사용한 용어는 단지 설명을 목적으로 사용된 것으로, 한정하려는 의도로 해석되어서는 안된다. 단수의 표현은 문맥상 명백하게 다르게 뜻하지 않는 한, 복수의 표현을 포함한다. 본 명세서에서, "포함하다" 또는 "가지다" 등의 용어는 명세서 상에 기재된 특징, 숫자, 단계, 동작, 구성요소, 부품 또는 이들을 조합한 것이 존재함을 지정하려는 것이지, 하나 또는 그 이상의 다른 특징들이나 숫자, 단계, 동작, 구성요소, 부품 또는 이들을 조합한 것들의 존재 또는 부가 가능성을 미리 배제하지 않는 것으로 이해되어야 한다.

<0022> 또한, 첨부 도면을 참조하여 설명함에 있어, 도면 부호에 관계없이 동일한 구성 요소는 동일한 참조부호를 부여하고 이에 대한 중복되는 설명은 생략하기로 한다. 실시예를 설명함에 있어서 관련된 공지 기술에 대한 구체적인 설명이 실시

예의 요지를 불필요하게 흐릴 수 있다고 판단되는 경우 그 상세한 설명을 생략한다.

<0023> 이하, 본 발명의 실시예를 첨부된 도면을 참조하여 상세하게 설명한다.

<0024> 도 1은 본 발명의 일실시예에 따른 연합 학습 시스템을 도시한 도면이다.

<0025> 본 발명의 일실시예에 따른 연합 학습 시스템은 클라우드 노드(110)와 엣지 노드(120)로 구성될 수 있다. 이때, 클라우드 노드(110)는 FL(Federated Learning) 클라우드 노드이고, 엣지 노드(120)는 FL 엣지 노드일 수 있다. 예를 들어, 본 발명의 일실시예에 따른 연합 학습 시스템은 쿠버네티스(KUBERNETES) 컨테이너 오케스트레이션 플랫폼으로 구현될 수 있다.

<0026> 엣지 노드(120)는 FL 애플리케이션의 실행을 관리하는 FL 포드를 포함할 수 있다. 또한, 클라우드 노드(110)는 엣지 노드(120)에 포함된 FL 포드의 리소스를 관리할 수 있다. 구체적으로, 클라우드 노드(110)는, FL 포드의 과거 리소스 사용률 정보에 기초하여 FL 포드의 권장 리소스 요청을 결정할 수 있다. 그리고, 클라우드 노드(110)는 FL 포드의 현재 리소스 요청과 권장 리소스 요청 간의 차이를 식별할 수 있다. 현재 리소스 요청과 권장 리소스 요청 간의 차이가 임계값을 초과하는 경우, 클라우드 노드(110)는 FL 포드의 리소스 요청 및 리소스 제한을 권장 리소스 요청에 따라 업데이트할 수 있다.

<0027> 본 발명의 일실시예에 따른 연합 학습 시스템은 시계열 과거 데이터(과거 리소스 사용률 정보)를 분석하고, 최적화 기법을 활용하여 실제 애플리케이션 수요에 따라 컨테이너 기반 애플리케이션 서비스에 할당된 리소스를 동적으로 조정함으로

써, 연합 학습의 정확도를 높이고 처리 속도를 개선할 수 있다.

<0028> 도 2는 본 발명의 일실시예에 따른 연합 학습 시스템의 클라우드 노드와 엣지 노드의 상세 구성 및 동작을 나타내는 도면이다.

<0029> 엣지 노드(120)는 도 2에 도시된 바와 같이 FL 포드(210), 및 VPA API 오브젝트(220)를 포함할 수 있다.

<0030> FL 포드(210)는 FL 애플리케이션의 실행을 관리하고, 리소스 할당을 처리할 수 있다. FL 포드(210)는 VPA 어드미션 플러그인(250)에 의하여 생성될 수 있다. VPA 어드미션 플러그인(250)은 FL 포드(210)를 생성할 때, 생성할 신규 FL 포드가 사용할 리소스의 량을 신규 FL 포드의 리소스 제한(initial limitation) 및 리소스 요청(initial request)으로 설정할 수 있다. 리소스 제한은 신규 FL 포드가 사용 가능한 최대 리소스량이고, 리소스 요청은 신규 FL 포드가 사용할 최소 리소스량일 수 있다. 따라서, 리소스 제한은 리소스 요청 이상이 되도록 설정될 수 있다. 또한, VPA 어드미션 플러그인(250)은 여유 리소스가 리소스 요청 이상인 엣지 노드(120)에 신규 FL 포드를 생성할 수 있다. FL 포드(210)가 필요로 하는 리소스는 CPU, 및 RAM과 같은 메모리이므로, 리소스 요청은 CPU 요청(CPU Request), 메모리 요청(Memory Request), 또는 RAM 요청(RAM Request) 중 하나이고, 리소스 제한은 CPU 제한(CPU Limit), 메모리 제한(Memory Limit) 또는 RAM 제한(RAM Limit)중 하나일 수 있다.

<0031> VPA API 오브젝트(220)는 FL 포드(210)에서 VPA(Vertical Pod Autoscaler) 동작을 구성하고, FL 포드(210)를 모니터링(221)하여 FL 포드(210)의 상태 정보를

관리할 수 있다. 예를 들어 FL 포드(210)의 상태 정보는 수직 자동 확장 정책에 필요한 정보이며, FL 포드(210)의 리소스 상태에 대한 정보를 포함할 수 있다.

<0032> 클라우드 노드(110)는 매트릭 서버(230), 히스토리 스토리지(240), VPA 어드미션 플러그인(250) 및 VPA 컨트롤러(260)를 포함할 수 있다.

<0033> 매트릭 서버(Metrics server)(230)는 FL 포드(210)에서 FL 포드(210)의 리소스 정보를 수집(211)할 수 있다. 예를 들어, FL 포드(210)의 리소스 정보는 CPU 요청, 메모리 요청, 엣지 노드(120)의 리소스 사용률 데이터 또는 FL 포드(210)의 리소스 사용률 데이터 중 적어도 하나를 포함할 수 있다. 그리고, 매트릭 서버(230)는 수집한 리소스 정보를 시계열 데이터 구조를 사용하여 히스토리 스토리지(240)에 저장(232)할 수 있다. 또한, 매트릭 서버(230)는 히스토리 스토리지(240)에 저장하는 정보를 사용자, 또는 다른 포드가 쿼리하고 액세스할 수 있도록 설정함으로써, 사용자 또는 다른 포드가 리소스 할당 및 확장에 관한 결정을 내릴 수 있도록 할 수 있다.

<0034> 또한, 매트릭 서버(230)는 수집한 FL 포드(210)의 리소스 정보에서 현재 시점의 CPU 요청, 메모리 요청을 추출하여 FL 포드(210)의 현재 리소스 요청을 생성할 수 있다. 그리고, 매트릭 서버(230)는 FL 포드(210)의 현재 리소스 요청을 VPA 컨트롤러(260)로 전송(231)할 수 있다.

<0035> 히스토리 스토리지(240)는 매트릭 서버(230)에 의하여 시계열 데이터 구조로 저장된 리소스 정보들을 관리할 수 있다. 또한, 히스토리 스토리지(240)는 매트릭 서버(230)에 의하여 시계열 데이터 구조로 저장된 현재 시간 이전의 리소스 정보들

을 과거 리소스 사용률 정보로 저장할 수 있다. 과거 리소스 사용률 정보는 이전 시간의 CPU 사용량과 메모리 사용량을 포함한 이 기록 데이터이며, VPA 성능을 최적화하기 위하여 사용될 수 있다.

<0036> VPA는 과거 리소스 사용률 정보를 분석함으로써, 향후 리소스 수요를 예측하고, 최적화 전략을 개발하고, 이상 징후를 감지하고, 시간 경과에 따른 성능을 평가할 수 있다. 즉, 히스토리 스토리지(240)는 리소스 관리에서 데이터 기반의 의사 결정을 지원하여 엣지 컴퓨팅 환경의 효율성과 안정성을 향상시킬 수 있다.

<0037> VPA 어드미션 플러그인(VPA admission plugin)(250)는 VPA API 객체를 기반으로 신규 FL 포드 생성 요청에 권장 리소스 요청을 적용(251)할 수 있다. 구체적으로, VPA 어드미션 플러그인(250)는 VPA 컨트롤러(260)의 리커맨더(Recommender)로부터 권장 리소스 요청을 수신할 수 있다. 그리고, VPA 어드미션 플러그인(250)는 엣지 노드(120)에서 FL 포드(210)를 생성할 때, 생성할 신규 FL 포드의 리소스 요청 및 리소스 제한을 권장 리소스 요청에 따라 설정할 수 있다.

<0038> 또한, VPA 어드미션 플러그인(250)는 기존 FL 포드의 리소스 요청 및 리소스 제한을 권장 리소스 요청에 따라 업데이트할 수 있다. 그리고, VPA 어드미션 플러그인(250)은 엣지 노드(120)의 VPA API 오브젝트(220)의 리소스 상태를 모니터링할 수 있다.

<0039> VPA 컨트롤러(VPA Controller)(260)는 FL 포드(210)의 권장 리소스 요청을 결정할 수 있다. VPA 컨트롤러(260)는 엣지 노드(120)의 VPA API 오브젝트(220)의 리소스 상태를 모니터링하고, VPA API 오브젝트(220)의 리소스 상태를 업데이트

설정(Set recommendation)할 수 있다.

<0040> 또한, VPA 컨트롤러(260)는 하기의 세 가지 모드 중 하나로 VPA의 동작을 제어할 수 있다. 그리고, 본 명세서는 자동 모드가 수행되는 경우의 동작을 기본으로 설명한다.

<0041> 1) 끄기(Off) 모드: 끄기 모드가 수행되는 경우, VPA 컨트롤러(260)는 VPA를 비활성화할 수 있다. 다만, 리커맨더는 VPA 개체에서 권장 리소스를 설정할 수 있다. 그러나, FL 포드(210)는 리커맨더가 생성한 권장 리소스에 영향을 받지 않을 수 있다. 즉, 끄기 모드가 수행되는 경우, FL 포드(210)의 리소스는 변경되지 않을 수 있다.

<0042> 2) 초기(Initial) 모드: 초기 모드가 수행되는 경우, VPA 컨트롤러(260)의 제어에 따라 VPA는 새로 생성된 FL 포드(210)에 대하여 리커맨더가 생성한 권장 리소스 요청을 제공할 수 있다. 다만, VPA는 실행 중인 FL 포드(210)는 리소스를 조정하지 않을 수 있다. 즉, 초기 모드가 수행되는 경우, 엣지 노드(120)에서 신규로 생성되는 FL 포드(210)의 리소스는 권장 리소스 요청에 따라 설정되나, 기존에 실행 중이던 FL 포드(210)의 리소스는 변경되지 않을 수 있다.

<0043> 3) 자동(Auto) 모드: 자동 모드가 수행되는 경우, VPA 컨트롤러(260)의 제어에 따라 VPA가 지속적으로 FL 포드(210)를 모니터링할 수 있다. 그리고, VPA 컨트롤러(260)는 필요한 경우 실제 리소스 사용량에 따라 리소스 요청을 조정할 수 있다. 또한, FL 포드(210)의 리소스와 권장 리소스 요청과의 차이가 임계값을 초과하는 경우, VPA 컨트롤러(260)는 FL 포드(210)를 퇴출하고, 리소스 요청을 업데이

트하여 다시 생성하도록 강제할 수도 있다.

<0044> VPA 컨트롤러(260)는 리커맨더 및 업데이터를 포함할 수 있다.

<0045> 리커맨더는 FL 포드의 과거 리소스 사용률 정보에 기초하여 FL 포드의 권장 리소스 요청을 결정할 수 있다. 이때, 리커맨더는 매트릭 서버(230)로부터 FL 포드(210)의 현재 리소스 요청을 수신(231)할 수 있다. 그리고, 리커맨더는 히스토리 스토리지(240)에 저장된 과거 리소스 사용률 정보에 액세스(241)하여 과거 리소스 사용률 정보를 수집할 수도 있다. 다음으로, 리커맨더는 과거 리소스 사용률 정보와 FL 포드(210)의 현재 리소스 요청에 기초하여 최적의 권장 리소스 요청을 결정함으로써, 즉각적인 요구사항에 유연하게 대응하고 장기적인 리소스 추세를 고려한 권장 사항을 제공할 수 있다.

<0046> 업데이터(Updater)는 자동 모드로 동작하는 경우, FL 포드(210)를 최신 권장 사항으로 업데이트하도록 결정할 수 있다. 업데이터는 엣지 노드(120)의 모든 VPA API 오브젝트(220)들과 FL 포드(210)들을 주기적으로 모니터링(261, 262)하여 FL 포드의 현재 리소스 요청을 식별할 수 있다. 또한,업데이터는 리커맨더로부터 권장 리소스 요청을 수신할 수 있다. 이때, 업데이터는 FL 포드의 현재 리소스 요청과 권장 리소스 요청 간의 차이를 식별할 수 있다.

<0047> 현재 리소스 요청과 권장 리소스 요청 간의 차이가 임계값을 초과하는 경우, 업데이터는 FL 포드의 리소스 요청 및 리소스 제한을 권장 리소스 요청에 따라 업데이트할 수 있다. 이때, 업데이터는 이전에 있던 FL 포드(210)를 퇴출할 수 있다. FL 포드(210)가 퇴출된 경우, VPA 컨트롤러(260)는 FL 포드(210)의 종료를 감지하

고 FL 포드(210)를 다시 생성하기 위한 조치를 취할 수 있다. 이때, VPA 컨트롤러(260)에 의하여 엣지 노드(120)에 새로 생성된 신규 FL 포드(210)에는 VPA에서 제공하는 권장 리소스 요청이 적용됨으로써, 권장 리소스 요청과 현재 리소스 요청의 차이가 큰 FL 포드(210)를 권장 리소스 요청이 적용된 신규 FL 포드(210)로 업데이트할 수 잇다.

<0048> VPA 컨트롤러(260)는 FL 포드(210)의 실제 리소스 요구 사항에 맞춰 FL 포드(210)의 리소스를 조정함으로써, 리소스 활용을 최적화하고 FL 포드(210)가 효율적으로 실행되는 데 필요한 리소스를 갖출 수 있도록 보장할 수 있다. 특히, VPA 컨트롤러(260)는 배포 롤링 업데이트 메커니즘을 사용하여 새로 생성된 신규 FL 포드(210)가 처음부터 트레이닝 없이도 퇴출된 FL 포드(210)의 연합 학습 작업을 계속 수행하도록 할 수 있다.

<0049> 예를 들어, 본 발명의 일실시예에 따른 연합 학습 시스템은 하기와 같은 VPA 알고리즘을 수행할 수 있다.

Algorithm 1 VPA Algorithm

Input: $min_{CPU} = 1000$, $min_{RAM} = 2048$, $SafetyMargin(SM) = 0.15$, $ratio_{CPU} = \frac{iniLimitCPU}{iniRequestCPU}$, $ratio_{RAM} = \frac{iniLimitRAM}{iniRequestRAM}$.
Output: Recommend resources: $rec_{CPU,RAM}$.

procedure
  Retrieve the list of VPA API objects from all nodes
  $vpaList = [vpaEdge1, vpaEdge2, vpaEdge3, ...]$
  while len($vpaList$) >= 1 **do**
    for $FLpod \in vpaList$ **do**
      // Get the resource utilization history from Prometheus under time-series data:
      $historyDict$ = {
      - Timestamp VPA_API_object CPU RAM
      - 24/03/11 10:00 vpaEdge1 1.0 2.0
      - 24/03/11 10:01 vpaEdge1 1.1 2.3
      - ... (more data points) }
      // Compute the optimal resources for each pod:
      $rec_{CPU,RAM} \leftarrow$ **Recommender**($historyDict$)
      // Update the new resource to FL pod:
      $req_{CPU,RAM}, lim_{CPU,RAM} \leftarrow$ **Updater**($rec_{CPU,RAM}$)
    end for
    time.sleep(60s)
  end while
end procedure
procedure RECOMMENDER($historyDict$)
  Initialize parameter $target = 0.9$
  size = len($historyDict$)
  $historyList \leftarrow historyDict$[key=CPU,key=RAM]
  $sortHistoryList \leftarrow$ sort($historyList$, ascending = True)
  $targetEstimate_{CPU,RAM} \leftarrow sortHistoryList$[size*$target$]
  $targetEstimate_{CPU,RAM} += targetEstimate_{CPU,RAM} * SM$
  // Set $minResource$ in the absence of historical data:
  $minResources_{CPU,RAM} \leftarrow min_{CPU}, min_{RAM}$
  $rec_{CPU,RAM} \leftarrow$ max($minResources, targetEstimate$)
  return $rec_{CPU,RAM}$
end procedure
procedure UPDATER(rec_{CPU}, rec_{RAM})
  Initialize parameter $threshold = 0.1$
  $req_{CPU,RAM} \leftarrow getCurReqValue(FLpod)$
  $diff_{CPU,RAM} \leftarrow \frac{|req_{CPU,RAM} - rec_{CPU,RAM}|}{req_{CPU,RAM}}$
  if $diff_{CPU} > threshold$ or $diff_{RAM} > threshold$ **then**
    $req_{CPU,RAM} \leftarrow rec_{CPU,RAM}$
    $lim_{CPU,RAM} \leftarrow rec_{CPU} * ratio_{CPU}, rec_{RAM} * ratio_{RAM}$
  end if
  return $req_{CPU,RAM}, lim_{CPU,RAM}$
end procedure

<0050>

<0051> 상기 VPA 알고리즘에서는 입력 파라미터를 고려하여 FL 포드에 필요한 최소 권장 CPU와 RAM(minCPU, minRAM)을 각각 CPU 코어 1개와 2GB 메모리로 설정할 수 있다. 이때, 두 파라미터는 권장 모듈에 대한 초기 VPA 권장설정으로도 사용된다. 또한, 안전 마진 (SM, Safety Margin)은 일반적으로 권장 리소스 값에 안전 또는 유연성 마진을 도입하는 데 사용될 수 있다. 그리고, 초기 리소스 제한과 초기 리소스 요청 간의 비율 (ratioCPU, ratioRAM)는 포드에 대해 설정된 초기 리소스 제한 (initial limitation)과 해당 포드의 초기 리소스 요청 (initial request) 간

의 비율을 정의한다. 또한, VPA 알고리즘의 출력값은 FL 포드에 CPU 및 RAM에 대한 최적화된 리소스 권장설정 (recCPU,RAM) 결과일 수 있다.

<0052> 도 3은 본 발명의 일실시예에 따른 과거 리소스 사용률 정보 검색 과정의 조일례이다.

<0053> 단계(310)에서 VPA 컨트롤러(260)가 히스토리 스토리지(240)에서 FL 포드의 과거 리소스 사용률 정보를 검색할 수 있다.

<0054> 단계(320)에서 VPA 컨트롤러(260)는 검색한 과거 리소스 사용률 정보에 기초하여 FL 포드의 권장 리소스 요청을 결정할 수 있다. 이때, 단계(320)는 상기 VPA 알고리즘의 라인 4-8의 for루프에 대응할 수 있다.

<0055> 구체적으로, VPA 컨트롤러(260)의 리커맨더에 포함된 Recommender() 함수는 과거 리소스 사용률 정보에 기초하여 권장 리소스 요청에 포함되는 권장 CPU 및 RAM 리소스를 결정할 수 있다. 또한, VPA 컨트롤러(260)의 업데이터에 포함된 Updater()함수는 현재 리소스 요청과 권장 리소스 요청 간의 차이가 임계값을 초과하는 경우, FL 포드의 리소스 요청 및 리소스 제한을 권장 리소스 요청에 따라 업데이트할 수 있다. 리커맨더(Recommender) 함수, 및 업데이터(Updater) 함수의 구체적인 구성 및 동작은 이하 도 4를 참조하여 설명한다.

<0056> VPA 컨트롤러(260)는 연합 학습 시스템의 리스트에 포함된 FL 포드(210)들 각각을 처리하고 리소스 할당을 업데이트한 후, for 루프를 완료할 수 있다.

<0057> 이때, VPA 컨트롤러(260)는 루프의 다음 반복을 시작하기 전에 60초 일시 중지를 설정할 수 있다. 일시 중지는 주기적인 리소스 조정 및 모니터링을 허용하기

위한 VPA 작동의 기본 시간 간격이며, 각 애플리케이션에 따라 변경할 수 있다.

<0058> 본 발명의 일실시예에 따른 연합 학습 시스템은 시계열 과거 데이터(과거 리소스 사용률 정보)를 분석하고, 최적화 기법을 활용하여 실제 애플리케이션 수요에 따라 컨테이너 기반 애플리케이션 서비스에 할당된 리소스를 동적으로 조정함으로써, 엣지 노드(120)의 리소스 사용률을 최적화할 수 있다.

<0059> 상기 VPA 알고리즘은 리소스 모니터링 및 조정 작업을 처리하기에 충분한 컴퓨팅 리소스가 있는 클라우드 노드(110)에서 실행되도록 설계될 수 있다.

<0060> 클라우드 노드(110)는 매트릭 서버(230)를 통해 엣지 노드(120)의 리소스 상태를 모니터링하고, 엣지 노드(120)에 통합된 VPA API 오브젝트(220)를 통해 각 포드 리소스에 대한 VPA를 결정할 수 있다. VPA 알고리즘에서 수행하는 계산 작업의 대부분이 클라우드 노드(110)에 할당되므로, 제안된 VPA 알고리즘은 계산 기능이 제한되고 리소스 제약이 있는 장치를 포함하여 다양한 유형의 엣지 노드(120)에서 작동할 수 있다.

<0061> 도 4는 본 발명의 일실시예에 따른 리커맨더 함수, 및 업데이터 함수의 일례이다.

<0062> 리커맨더(Recommender) 함수(410)는 먼저 목표 백분위수(target)를 0.9로 초기화할 수 있다. 그리고, 히스토리 스토리지(240)의 기록 데이터(historyDict)에서 CPU 및 RAM 정보를 추출하여 오름차순으로 정렬할 수 있다.(sortHistoryList).

<0063> 다음으로, 리커맨더 함수(410)는 지정된 목표 백분위수에서 기록 리소스 값을 선택하고 SM (Safety Margin)을 곱하여 목표 추정치(targetEstimateCPU,RAM)를

결정할 수 있다.

<0064> 히스토리 스토리지(240)에서 과거 데이터를 추출할 수 없는 경우, 리커맨더 함수(410)는 최소 리소스(minCPU, RAM)를 최소 CPU 및 RAM 값으로 설정할 수 있다.

<0065> 즉, 리커맨더 함수(410)는 최소 리소스 및 목표추정치 중 높은 값을 권장 리소스 요청의 권장 설정(recCPU,RAM) 값으로 결정하여 리턴할 수 있다.

<0066> 업데이터(Updater) 함수(420)는 임계값을 0.1로 초기화할 수 있다. 다음으로, 업데이터 함수(420)는 FL 포드의 현재 리소스 요청(reqCPU,RAM)을 검색할 수 있다. 그 다음으로, 업데이터 함수(420)는 CPU와 RAM의 현재 값과 권장 값 사이의 상대적 차이(diffCPU,RAM)를 계산할 수 있다.

<0067> 상대적 차이(diffCPU,RAM)가 지정된 임계값을 초과하는 경우, 업데이터 함수(420)는 리소스 요청값과 리소스 한계값을 권장값(recCPU,RAM)으로 업데이트할 수 있다.

<0068> 업데이터 함수(420)는 현재 값과 권장 값의 차이가 정의된 임계값을 초과하는 경우에만 리소스 요청이 조정되도록 함으로써, 그 차이가 허용 범위 내에 있는 경우에는 불필요한 조정을 방지할 수 있다.

<0069> 도 5는 본 발명의 일실시예에 따른 연합 학습 방법을 도시한 플로우차트이다.

<0070> 단계(510)에서 VPA 컨트롤러(260)는 히스토리 스토리지(240)에 접속하여 FL 포드(210)의 과거 리소스 사용률 정보를 검색할 수 있다.

<0071> 단계(520)에서 VPA 컨트롤러(260)는 단계(510)에서 검색한 과거 리소스 사용

률 정보에 기초하여 FL 포드(210)의 권장 리소스 요청을 결정할 수 있다.

<0072> 단계(530)에서 VPA 컨트롤러(260)는 엣지 노드(120)의 VPA API 오브젝트(220)의 리소스 상태를 모니터링하여 FL 포드(210)의 현재 리소스 요청을 측정하거나, 매트릭 서버(230)로부터 FL 포드(210)의 현재 리소스 요청을 수신할 수 있다.

<0073> 단계(540)에서 VPA 컨트롤러(260)는 단계(530)에서 측정한 FL 포드(210)의 현재 리소스 요청과 단계(520)에서 결정한 권장 리소스 요청 간의 차이를 식별할 수 있다.

<0074> 단계(550)에서 VPA 컨트롤러(260)는 현재 리소스 요청과 권장 리소스 요청 간의 차이가 임계값을 초과하는지 여부를 확인할 수 있다. 현재 리소스 요청과 권장 리소스 요청 간의 차이가 임계값을 초과하는 경우, VPA 컨트롤러(260)는 단계(560)를 수행할 수 있다. 현재 리소스 요청과 권장 리소스 요청 간의 차이가 임계값 이하인 경우, VPA 컨트롤러(260)는 현재 리소스 요청과 권장 리소스 요청 간의 차이가 임계값을 초과할 때까지 FL 포드(210)의 리소스 요청 및 리소스 제한을 업데이트하지 않고, 단계(510) 내지 단계(550)를 수행할 수 있다.

<0075> 단계(560)에서 VPA 컨트롤러(260)는 FL 포드(210)의 리소스 요청 및 리소스 제한을 단계(520)에서 결정한 권장 리소스 요청에 따라 업데이트할 수 있다.

<0076> 도 6은 본 발명의 일실시예에 따른 연합 학습 방법의 권장 리소스 요청 결정 과정을 도시한 플로우차트이다. 도 6의 단계(610) 내지 단계(660)는 도 5의 단계(520)에 포함될 수 있다.

<0077> 단계(610)에서 VPA 컨트롤러(260)는 히스토리 스토리지(240)의 기록 데이터(historyDict)에서 리소스 정보(CPU 및 RAM 정보)를 추출하여 오름차순으로 정렬할 수 있다. 이때, 정렬된 리소스 정보가 과거 리소스 사용률 정보 데이터일 수 있다.

<0078> 단계(620)에서 VPA 컨트롤러(260)는 단계(610)에서 정렬된 리소스 정보를 기초로 목표 추정치를 결정할 수 있다. 이때, VPA 컨트롤러(260)는 기 지정된 목표 백분위수에서 기록 리소스값을 선택할 수 있다. 그리고, VPA 컨트롤러(260)는 선택한 기록 리소스값과 안전 마진을 연산하여 목표 추정치를 결정할 수 있다.

<0079> 단계(630)에서 VPA 컨트롤러(260)는 과거 리소스 사용률 정보에 기초하여 최소 리소스를 결정할 수 있다. 이때, VPA 컨트롤러(260)는 과거 리소스 사용률 정보에서 가장 낮은 CPU값 및 RAM값을 검색할 수 있다. 그리고, VPA 컨트롤러(260)는 검색한 CPU값 및 RAM값을 최소 리소스로 결정할 수 있다.

<0080> 단계(640)에서 VPA 컨트롤러(260)는 목표 추정치가 최소 리소스를 초과하는지 여부를 확인할 수 있다. 목표 추정치가 최소 리소스를 초과하는 경우, VPA 컨트롤러(260)는 단계(650)를 수행할 수 있다. 목표 추정치가 최소 리소스 이하인 경우, VPA 컨트롤러(260)는 단계(660)를 수행할 수 있다.

<0081> 단계(650)에서 VPA 컨트롤러(260)는 단계(620)에서 결정된 목표 추정치를 권장 리소스 요청의 권장 설정으로 결정할 수 있다.

<0082> 단계(660)에서 VPA 컨트롤러(260)는 단계(630)에서 결정된 최소 리소스를 권장 리소스 요청의 권장 설정으로 결정할 수 있다.

<0083> 본 발명은 시계열 과거 데이터(과거 리소스 사용률 정보)를 분석하고, 최적화 기법을 활용하여 실제 애플리케이션 수요에 따라 컨테이너 기반 애플리케이션 서비스에 할당된 리소스를 동적으로 조정함으로써, 연합 학습의 정확도를 높이고 처리 속도를 개선할 수 있다.

<0084> 한편, 본 발명에 따른 연합 학습 장치 또는 연합 학습 방법은 컴퓨터에서 실행될 수 있는 프로그램으로 작성되어 마그네틱 저장매체, 광학적 판독매체, 디지털 저장매체 등 다양한 기록 매체로도 구현될 수 있다.

<0085> 본 명세서에 설명된 각종 기술들의 구현들은 디지털 전자 회로조직으로, 또는 컴퓨터 하드웨어, 펌웨어, 소프트웨어로, 또는 그들의 조합들로 구현될 수 있다. 구현들은 데이터 처리 장치, 예를 들어 프로그램가능 프로세서, 컴퓨터, 또는 다수의 컴퓨터들의 동작에 의한 처리를 위해, 또는 이 동작을 제어하기 위해, 컴퓨터 프로그램 제품, 예를 들어 기계 판독가능 저장 장치(컴퓨터 판독가능 매체)에서 유형적으로 구체화된 컴퓨터 프로그램으로서 구현될 수 있다. 상술한 컴퓨터 프로그램(들)과 같은 컴퓨터 프로그램은 컴파일된 또는 인터프리트된 언어들을 포함하는 임의의 형태의 프로그래밍 언어로 기록될 수 있고, 독립형 프로그램으로서 또는 모듈, 구성요소, 서브루틴, 또는 컴퓨팅 환경에서의 사용에 적절한 다른 유닛으로서 포함하는 임의의 형태로 전개될 수 있다. 컴퓨터 프로그램은 하나의 사이트에서 하나의 컴퓨터 또는 다수의 컴퓨터들 상에서 처리되도록 또는 다수의 사이트들에 걸쳐 분배되고 통신 네트워크에 의해 상호 연결되도록 전개될 수 있다.

<0086> 컴퓨터 프로그램의 처리에 적절한 프로세서들은 예로서, 범용 및 특수 목적

마이크로프로세서들 둘 다, 및 임의의 종류의 디지털 컴퓨터의 임의의 하나 이상의 프로세서들을 포함한다. 일반적으로, 프로세서는 판독 전용 메모리 또는 랜덤 액세스 메모리 또는 둘 다로부터 명령어들 및 데이터를 수신할 것이다. 컴퓨터의 요소들은 명령어들을 실행하는 적어도 하나의 프로세서 및 명령어들 및 데이터를 저장하는 하나 이상의 메모리 장치들을 포함할 수 있다. 일반적으로, 컴퓨터는 데이터를 저장하는 하나 이상의 대량 저장 장치들, 예를 들어 자기, 자기-광 디스크들, 또는 광 디스크들을 포함할 수 있거나, 이것들로부터 데이터를 수신하거나 이것들에 데이터를 송신하거나 또는 양쪽으로 되도록 결합될 수도 있다. 컴퓨터 프로그램 명령어들 및 데이터를 구체화하는데 적절한 정보 캐리어들은 예로서 반도체 메모리 장치들, 예를 들어, 하드 디스크, 플로피 디스크 및 자기 테이프와 같은 자기 매체(Magnetic Media), CD-ROM(Compact Disk Read Only Memory), DVD(Digital Video Disk)와 같은 광 기록 매체(Optical Media), 플롭티컬 디스크(Floptical Disk)와 같은 자기-광 매체(Magneto-Optical Media), 롬(ROM, Read Only Memory), 램(RAM, Random Access Memory), 플래시 메모리, EPROM(Erasable Programmable ROM), EEPROM(Electrically Erasable Programmable ROM) 등을 포함한다. 프로세서 및 메모리는 특수 목적 논리 회로조직에 의해 보충되거나, 이에 포함될 수 있다.

<0087> 또한, 컴퓨터 판독가능 매체는 컴퓨터에 의해 액세스될 수 있는 임의의 가용 매체일 수 있고, 컴퓨터 저장매체를 모두 포함할 수 있다.

<0088> 본 명세서는 다수의 특정한 구현물의 세부사항들을 포함하지만, 이들은 어떠한 발명이나 청구 가능한 것의 범위에 대해서도 제한적인 것으로서 이해되어서는

안되며, 오히려 특정한 발명의 특정한 실시형태에 특유할 수 있는 특징들에 대한 설명으로서 이해되어야 한다. 개별적인 실시형태의 문맥에서 본 명세서에 기술된 특정한 특징들은 단일 실시형태에서 조합하여 구현될 수도 있다. 반대로, 단일 실시형태의 문맥에서 기술한 다양한 특징들 역시 개별적으로 혹은 어떠한 적절한 하위 조합으로도 복수의 실시형태에서 구현 가능하다. 나아가, 특징들이 특정한 조합으로 동작하고 초기에 그와 같이 청구된 바와 같이 묘사될 수 있지만, 청구된 조합으로부터의 하나 이상의 특징들은 일부 경우에 그 조합으로부터 배제될 수 있으며, 그 청구된 조합은 하위 조합이나 하위 조합의 변형물로 변경될 수 있다.

<0089> 마찬가지로, 특정한 순서로 도면에서 동작들을 묘사하고 있지만, 이는 바람직한 결과를 얻기 위하여 도시된 그 특정한 순서나 순차적인 순서대로 그러한 동작들을 수행하여야 한다거나 모든 도시된 동작들이 수행되어야 하는 것으로 이해되어서는 안 된다. 특정한 경우, 멀티태스킹과 병렬 프로세싱이 유리할 수 있다. 또한, 상술한 실시형태의 다양한 장치 컴포넌트의 분리는 그러한 분리를 모든 실시형태에서 요구하는 것으로 이해되어서는 안되며, 설명한 프로그램 컴포넌트와 장치들은 일반적으로 단일의 소프트웨어 제품으로 함께 통합되거나 다중 소프트웨어 제품에 패키징 될 수 있다는 점을 이해하여야 한다.

<0090> 한편, 본 명세서와 도면에 개시된 본 발명의 실시 예들은 이해를 돕기 위해 특정 예를 제시한 것에 지나지 않으며, 본 발명의 범위를 한정하고자 하는 것은 아니다. 여기에 개시된 실시 예들 이외에도 본 발명의 기술적 사상에 바탕을 둔 다른 변형 예들이 실시 가능하다는 것은, 본 발명이 속하는 기술분야에서 통상의 지식을

가진 자에게 자명한 것이다.

【부호의 설명】

<0091> 110: 클라우드 노드

120: 엣지 노드

210: FL 포드

220: VPA API 오브젝트

230: 매트릭 서버

250: VPA 어드미션 플러그인

260: VPA 컨트롤러

【청구범위】

【청구항 1】

FL(Federated Learning) 포드의 과거 리소스 사용률 정보에 기초하여 상기 FL 포드의 권장 리소스 요청을 결정하는 단계;

상기 FL 포드의 현재 리소스 요청과 상기 권장 리소스 요청 간의 차이를 식별하는 단계; 및

상기 현재 리소스 요청과 상기 권장 리소스 요청 간의 차이가 임계값을 초과하는 경우, 상기 FL 포드의 리소스 요청 및 리소스 제한을 상기 권장 리소스 요청에 따라 업데이트하는 단계

를 포함하는 연합 학습 방법.

【청구항 2】

제1항에 있어서,

상기 권장 리소스 요청을 결정하는 단계는,

리소스 정보를 정렬하는 단계;

정렬된 리소스 정보를 기초로 목표 추정치를 결정하는 단계;

상기 과거 리소스 사용률 정보에 기초하여 최소 리소스를 결정하는 단계;

상기 목표 추정치가 최소 리소스를 초과하는 경우, 상기 목표 추정치를 상기 권장 리소스 요청의 권장 설정으로 결정하는 단계; 및

상기 목표 추정치가 최소 리소스 이하인 경우, 상기 최소 리소스를 상기 권장 리소스 요청의 권장 설정으로 결정하는 단계

를 포함하는 연합 학습 방법.

【청구항 3】

제2항에 있어서,

상기 목표 추정치를 결정하는 단계는,

기 지정된 목표 백분위수에서 기록 리소스값을 선택하는 단계; 및

선택한 기록 리소스값과 안전 마진을 연산하여 상기 목표 추정치를 결정하는 단계

를 포함하는 연합 학습 방법.

【청구항 4】

제2항에 있어서,

상기 최소 리소스를 결정하는 단계는,

상기 과거 리소스 사용률 정보에서 가장 낮은 CPU값 및 RAM값을 검색하는 단계; 및

검색한 CPU값 및 RAM값을 최소 리소스로 결정하는 단계

를 포함하는 연합 학습 방법.

【청구항 5】

FL 애플리케이션의 실행을 관리하는 FL 포드가 포함된 FL 엣지 노드; 및

상기 FL 포드의 리소스를 관리하는 FL 클라우드 노드

를 포함하고,

상기 FL 클라우드 노드는,

상기 FL 포드의 과거 리소스 사용률 정보에 기초하여 상기 FL 포드의 권장 리소스 요청을 결정하고, 상기 FL 포드의 현재 리소스 요청과 상기 권장 리소스 요청 간의 차이를 식별하며, 상기 현재 리소스 요청과 상기 권장 리소스 요청 간의 차이가 임계값을 초과하는 경우, 상기 FL 포드의 리소스 요청 및 리소스 제한을 상기 권장 리소스 요청에 따라 업데이트하는 연합 학습 시스템.

【청구항 6】

제5항에 있어서,

상기 FL 엣지 노드는,

상기 FL 애플리케이션의 실행을 관리하고, 리소스 할당을 처리하는 FL 포드 및

상기 FL 포드에서 VPA(Vertical Pod Autoscaler) 동작을 구성하고, 상기 FL 포드를 모니터링하여 상기 FL 포드의 상태 정보를 관리하는 VPA API 오브젝트

를 포함하는 연합 학습 시스템.

【청구항 7】

제5항에 있어서,

상기 FL 클라우드 노드는,

상기 FL 포드에서 상기 FL 포드의 리소스 정보를 수집하여 히스토리 스토리지에 저장하는 매트릭 서버;

저장된 리소스 정보들을 과거 리소스 사용률 정보로 나타내는 히스토리 스토리지;

VPA API 객체를 기반으로 신규 FL 포드 생성 요청에 상기 권장 리소스 요청을 적용하거나, 기존 FL 포드의 리소스 요청 및 리소스 제한을 상기 권장 리소스 요청에 따라 업데이트하는 VPA 어드미션 플러그인; 및

상기 FL 포드의 권장 리소스 요청을 결정하는 VPA 컨트롤러

를 포함하는 연합 학습 방법.

【청구항 8】

제7항에 있어서,

상기 VPA 컨트롤러는,

상기 FL 엣지 노드의 VPA API 오브젝트의 리소스 상태를 모니터링하고, 상기 VPA API 오브젝트의 리소스 상태를 업데이트 설정(Set recommendation)하는 연합 학습 시스템.

【청구항 9】

제7항에 있어서,

상기 VPA 어드미션 플러그인은,

상기 FL 엣지 노드의 VPA API 오브젝트의 리소스 상태를 모니터링하는 연합 학습 시스템.

【요약서】

【요약】

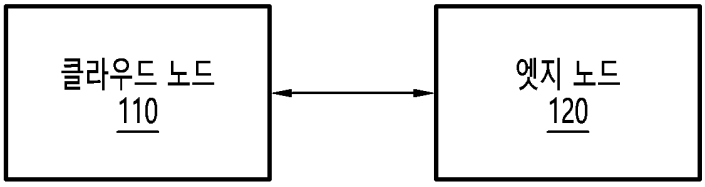
수직적 포드 오토 스케일러를 사용한 탄력적 연합 학습 방법 및 시스템이 개시된다. 연합 학습 방법은 FL 포드의 과거 리소스 사용률 정보에 기초하여 상기 FL 포드의 권장 리소스 요청을 결정하는 단계; 상기 FL 포드의 현재 리소스 요청과 상기 권장 리소스 요청 간의 차이를 식별하는 단계; 및 상기 현재 리소스 요청과 상기 권장 리소스 요청 간의 차이가 임계값을 초과하는 경우, 상기 FL 포드의 리소스 요청 및 리소스 제한을 상기 권장 리소스 요청에 따라 업데이트하는 단계를 포함할 수 있다.

【대표도】

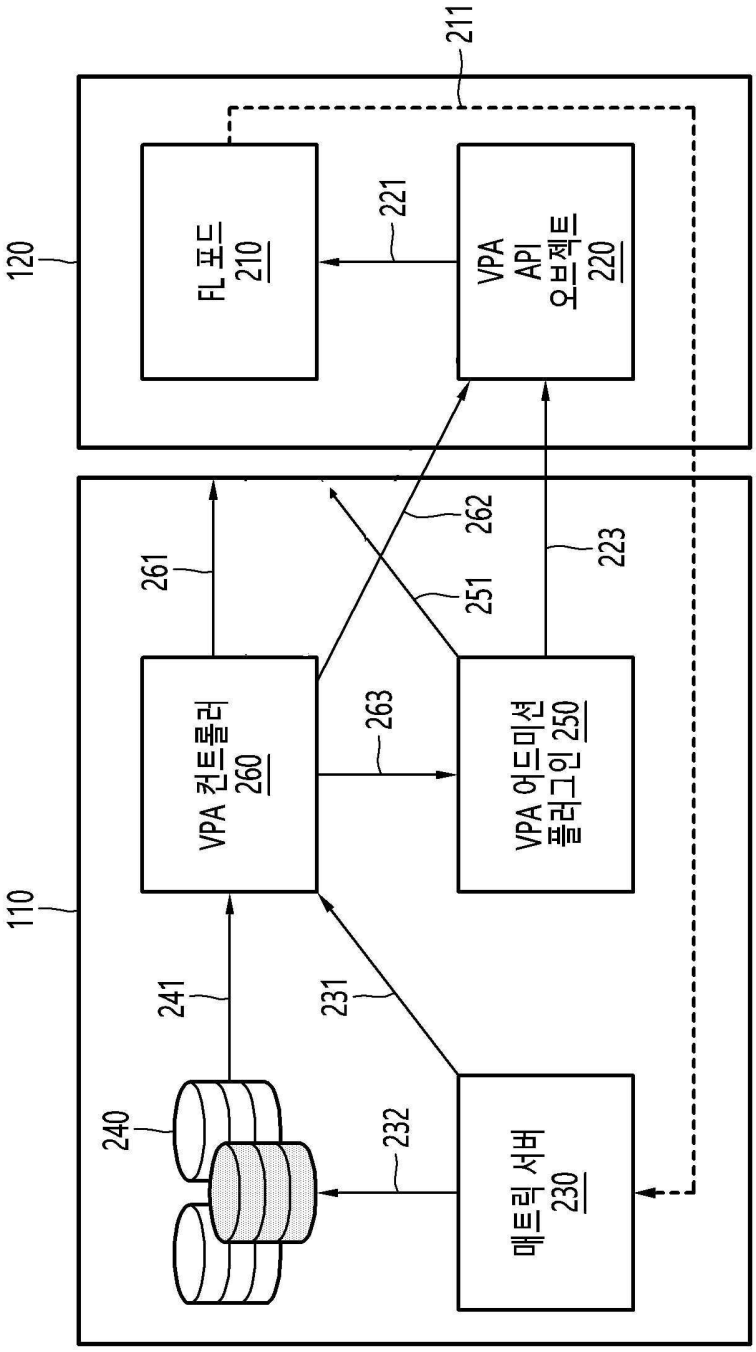
도 2

【도면】

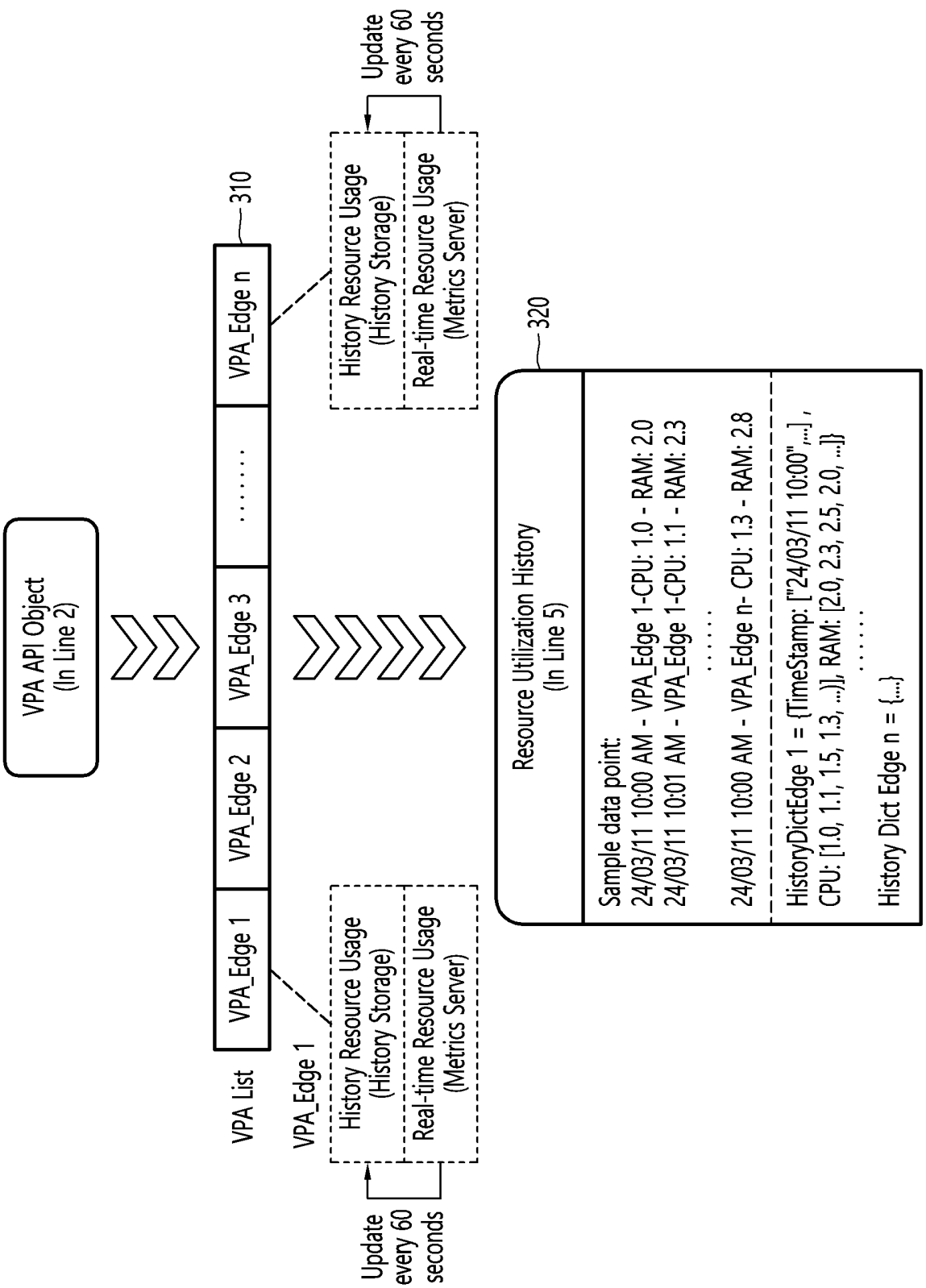
【도 1】



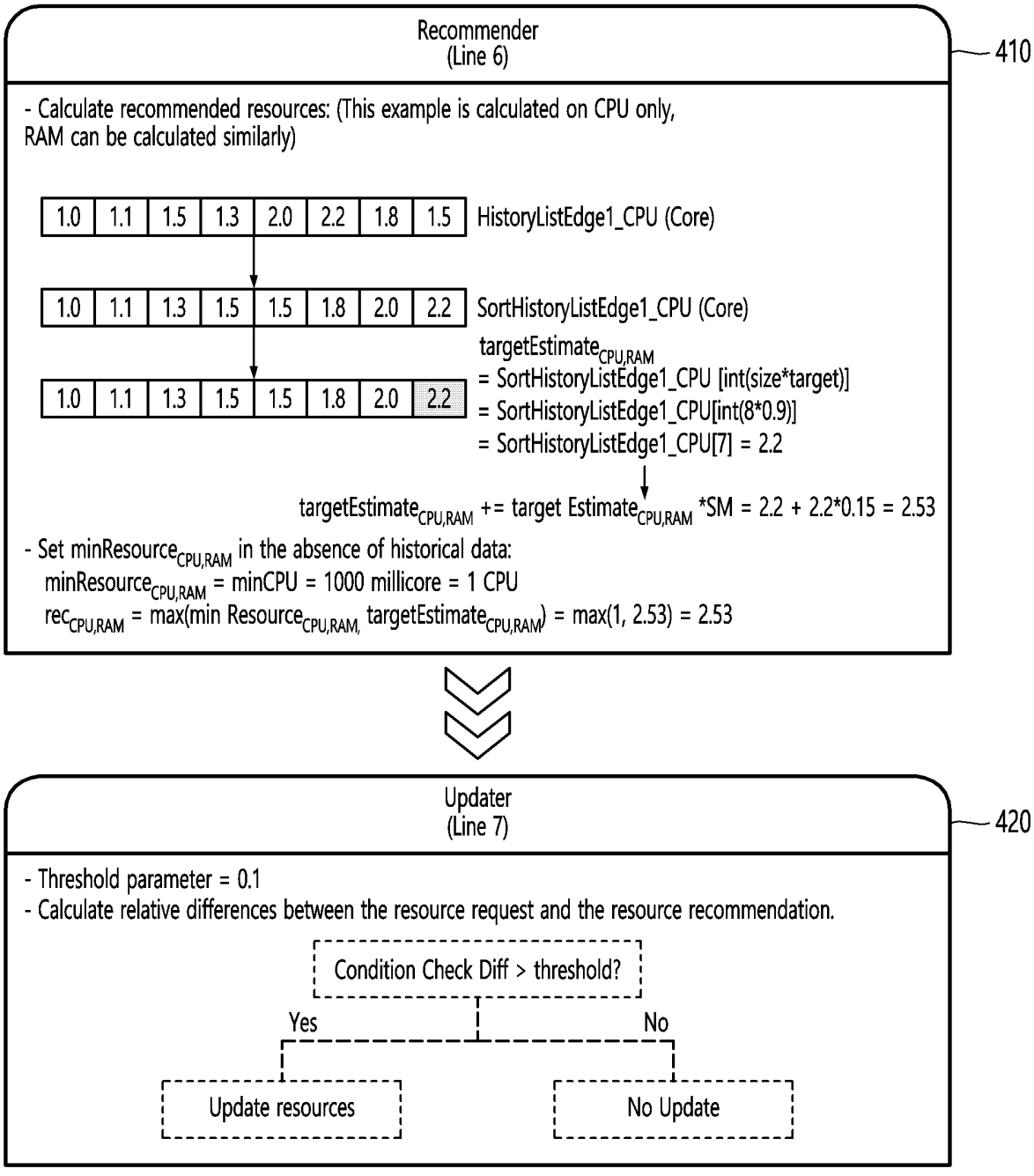
【도 2】


211
120
FL 포드
210
221
VPA
API
오브젝트
220
110
261
262
251
223
VPA 컨트롤러
260
263
VPA 어드미션
플러그인 250
241
231
240
232
메트릭 서버
230

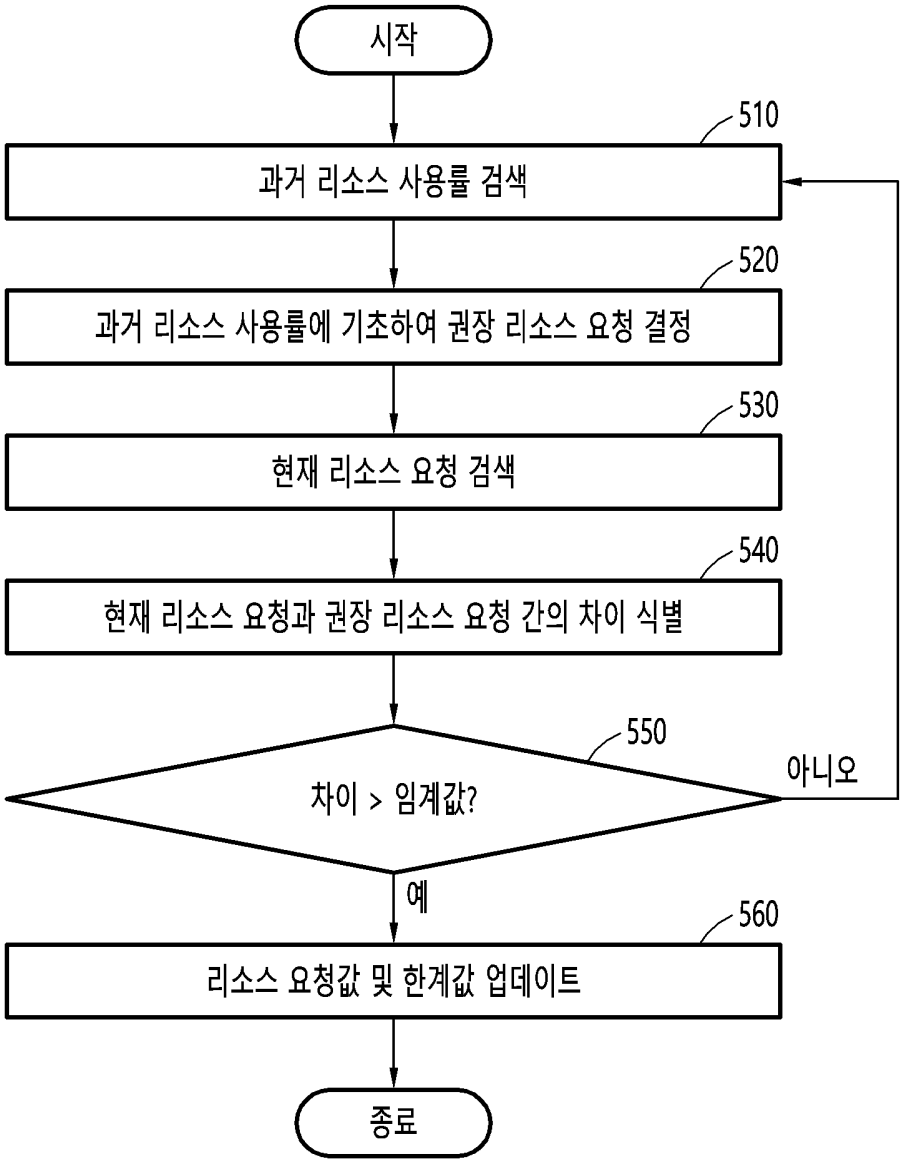
【도 3】


VPA API Object
(In Line 2)
VPA List
VPA_Edge 1
VPA_Edge 2
VPA_Edge 3
.....
VPA_Edge n
310
VPA_Edge 1
History Resource Usage
(History Storage)
Real-time Resource Usage
(Metrics Server)
Update
every 60
seconds
History Resource Usage
(History Storage)
Real-time Resource Usage
(Metrics Server)
Update
every 60
seconds
320
Resource Utilization History
(In Line 5)
Sample data point:
24/03/11 10:00 AM - VPA_Edge 1-CPU: 1.0 - RAM: 2.0
24/03/11 10:01 AM - VPA_Edge 1-CPU: 1.1 - RAM: 2.3
.....
24/03/11 10:00 AM - VPA_Edge n- CPU: 1.3 - RAM: 2.8
HistoryDictEdge 1 = {TimeStamp: ["24/03/11 10:00",...] ,
CPU: [1.0, 1.1, 1.5, 1.3, ...)], RAM: [2.0, 2.3, 2.5, 2.0, ..]}
.....
History Dict Edge n = {....}

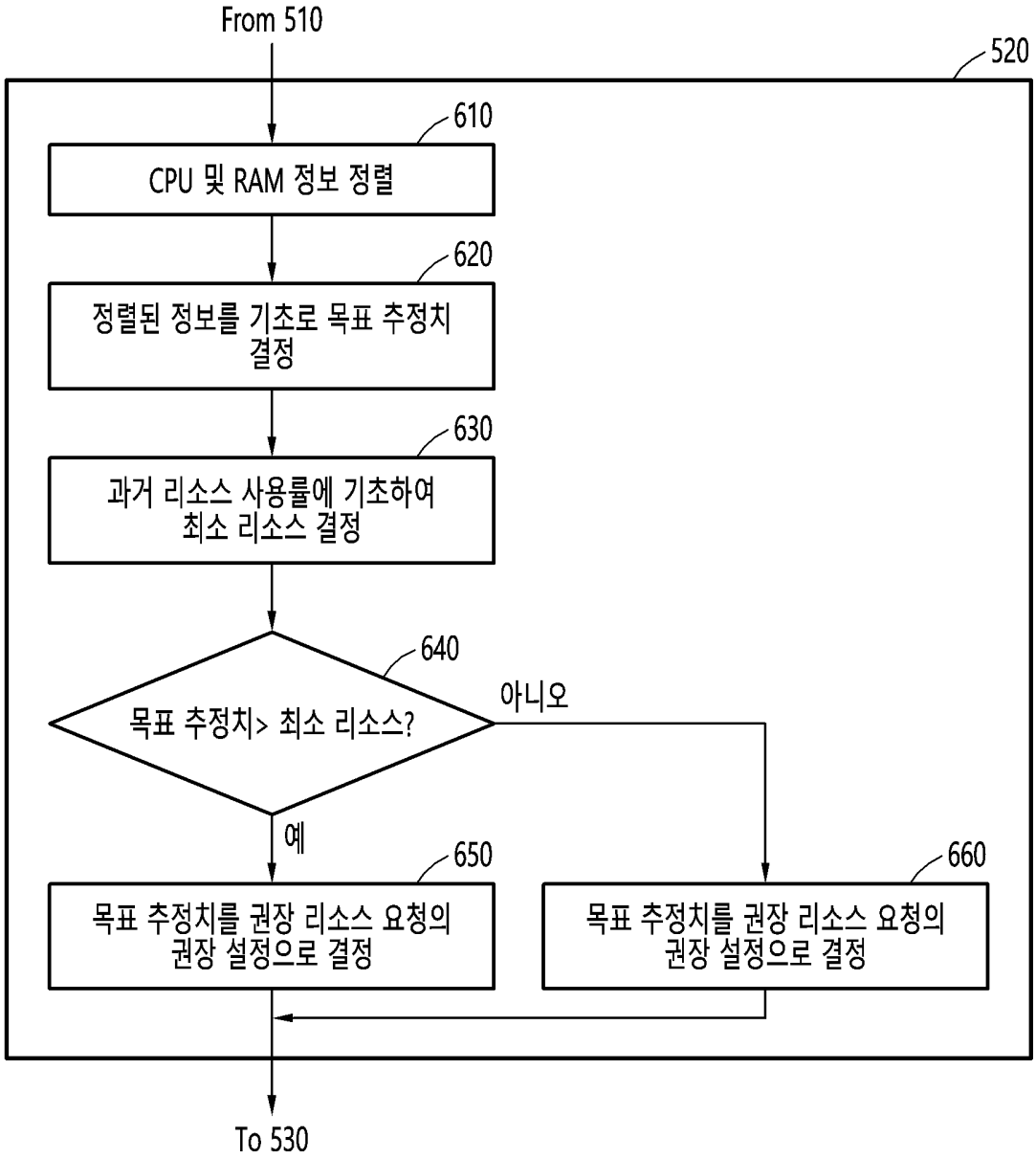
【도 4】


Recommender
(Line 6)
410
- Calculate recommended resources: (This example is calculated on CPU only,
RAM can be calculated similarly)
1.0 1.1 1.5 1.3 2.0 2.2 1.8 1.5
HistoryListEdge1_CPU (Core)
1.0 1.1 1.3 1.5 1.5 1.8 2.0 2.2
SortHistoryListEdge1_CPU (Core)
1.0 1.1 1.3 1.5 1.5 1.8 2.0 2.2
targetEstimate_CPU,RAM
= SortHistoryListEdge1_CPU [int(size*target)]
= SortHistoryListEdge1_CPU[int(8*0.9)]
= SortHistoryListEdge1_CPU[7] = 2.2
targetEstimate_CPU,RAM += target Estimate_CPU,RAM *SM = 2.2 + 2.2*0.15 = 2.53
- Set minResource_CPU,RAM in the absence of historical data:
minResource_CPU,RAM = minCPU = 1000 millicore = 1 CPU
rec_CPU,RAM = max(min Resource_CPU,RAM, targetEstimate_CPU,RAM) = max(1, 2.53) = 2.53
Updater
(Line 7)
420
- Threshold parameter = 0.1
- Calculate relative differences between the resource request and the resource recommendation.
Condition Check Diff > threshold?
Yes
No
Update resources
No Update

【도 5】


시작
510
과거 리소스 사용률 검색
520
과거 리소스 사용률에 기초하여 권장 리소스 요청 결정
530
현재 리소스 요청 검색
540
현재 리소스 요청과 권장 리소스 요청 간의 차이 식별
550
차이 > 임계값?
아니오
예
560
리소스 요청값 및 한계값 업데이트
종료

【도 6】


From 510
520
610
CPU 및 RAM 정보 정렬
620
정렬된 정보를 기초로 목표 추정치 결정
630
과거 리소스 사용률에 기초하여 최소 리소스 결정
640
목표 추정치> 최소 리소스?
아니오
예
650
목표 추정치를 권장 리소스 요청의 권장 설정으로 결정
660
목표 추정치를 권장 리소스 요청의 권장 설정으로 결정
To 530